



Examen Avaluació Ordinària		
Mòdul: Programació	Cicle:	Curs: 1er
Data d'examen: 20/05/2026		
Nom i Cognoms:		

Instruccions generals:

- Llig atentament tots els enunciats abans de respondre.
- Cada RA es qualifica sobre 10 punts: **2 punts** corresponen al **test** específic d'eixe RA i **8 punts** a la **part pràctica** indicada en cada exercici.

Part 1: Test per RAs (32 preguntes – 45 minuts)

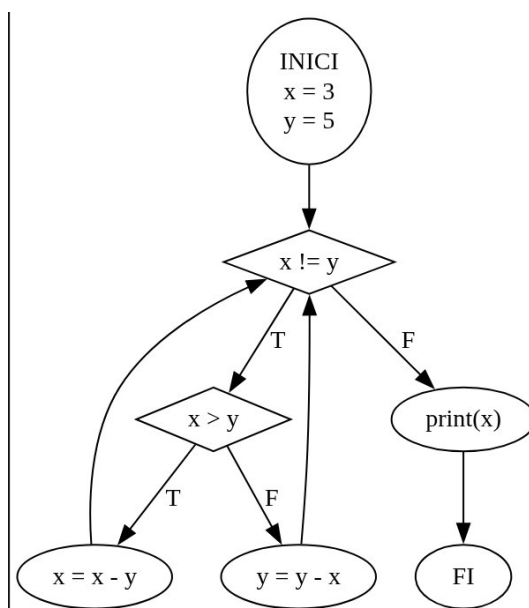
- Les respostes del test tenen una única opció correcta y **cada resposta incorrecta resta ½ de la puntuació d'una altra pregunta del mateix RA** (les respostes incorrectes d'un RA mai restaran puntuació d'un altre RA).
- **Poseu la resposta triada (a,b,c o d) als vuits de resposta associats a cada RA.** Si hi ha una resposta marcada al test pero no passada al vuit, es considerarà com a pregunta no contestada i no sumará ni restará puntuació.

RA1: Reconeix l'estructura d'un programa informàtic

1		2		3		4	
---	--	---	--	---	--	---	--

1. Què imprimirà el programa del següent diagrama de flux?

- a) 3
- b) 5
- c) 0
- d) 1





2. En Java, quin element determina el punt inicial d'execució d'una aplicació de consola?

- a) El primer constructor de la classe Main
- b) El primer mètode escrit dins del fitxer .java
- c) El mètode public static void main(String[] args)
- d) Qualsevol mètode marcat com public

3. Què mostra el següent fragment?

```
int a = 4;  
int b = 3;  
System.out.println(a * a % 3);
```

- a) 1
- b) 0
- c) 4
- d) 5

4. Quin és el problema principal del codi següent?

```
public class Prova {  
    public static void main(String[] args) {  
        int edat;  
        if (args.length > 0) {  
            edat = Integer.parseInt(args[0]);  
        }  
        System.out.println(edat);  
    }  
}
```

- a) args.length no es pot consultar
- b) Integer.parseInt només accepta enters, no cadenes
- c) La variable edat podria no haver sigut inicialitzada
- d) main no pot contindre condicionals



RA2. Escriu i prova programes senzills aplicant fonaments de POO

5		6		7		8	
---	--	---	--	---	--	---	--

5. Què fa aquest codi?

```
Random r = new Random(10);  
System.out.println(r.nextInt(5) + 1);
```

- a) Genera un enter entre 0 i 5 inclosos
- b) Genera un enter entre 1 i 5 inclosos
- c) Genera un decimal entre 1 i 5
- d) Genera sempre el mateix valor (5+1)

6. Quina afirmació és correcta?

```
String a = "java";  
String b = a.toUpperCase();  
System.out.println(b);  
System.out.println(a);
```

- a) Mostra JAVA i JAVA
- b) Mostra java i JAVA
- c) Mostra JAVA i java
- d) Dona error perquè toUpperCase() no pot aplicar-se a Strings

7. Quin conjunt de valors podria mostrar el programa?

```
String text = "java";  
System.out.println(text.charAt(2));
```

- a) a
- b) v
- c) j
- d) Error d'execució, perquè no hi ha cap lletra a la posició 2

8. Quina afirmació és correcta sobre objectes i classes?

- a) Una classe ocupa sempre una posició concreta en memòria com a dada final
- b) Un objecte és una instància concreta creada a partir d'una classe
- c) Tots els objectes d'una classe comparteixen sempre els mateixos valors d'atributs
- d) Una classe només pot tindre atributs, però no mètodes



RA3. Escriu i depura codi utilitzant estructures de control

9		10		11		12	
---	--	----	--	----	--	----	--

9. Quina eixida produeix aquest codi?

- a) -1
- b) -3
- c) -5
- d) 0

```
int suma = 0;

for (int i = 0; i <= 4; i++) {
    if (i / 2 == 0) {
        suma += 1;
    } else {
        suma -= 1;
    }
}

System.out.println(suma);
```

10. Quin és el problema del fragment següent?

```
int[] dades = {4, 7, 9};
for (int i = 0; i <= dades.length; i++) {
    System.out.println(dades[i]);
}
```

- a) El bucle no s'executa mai
- b) L'última iteració provoca un accés fora de rang
- c) length només funciona amb ArrayList, en arrays s'utilitza size
- d) Els arrays no poden recórrer-se amb for

11. Quina afirmació és correcta?

```
try {
    int n = Integer.parseInt("12a");
    System.out.println(n);
} catch (NumberFormatException e) {
    System.out.println("Format incorrecte");
} finally {
    System.out.println("Fi");
}
```



- a) Mostra només *Format incorrecte*
- b) Mostra *12a* i després *Fi*
- c) Mostra *Format incorrecte* i després *Fi*
- d) No compila perquè finalment només es pot usar si no hi ha catch

12. Què mostra el següent codi?

```
int n = 7;  
if (n > 0 || n % 2 == 0) {  
    System.out.println("A");  
} else {  
    System.out.println("B");  
}
```

- a) A
- b) B
- c) Imprimeix A i després B
- d) No compila

RA4. Desenvolupa programes organitzats en classes

13		14		15		16	
----	--	----	--	----	--	----	--

13. Quin problema té aquest codi?

```
class Producte {  
    private String nom;  
    public Producte(String nom) {  
        nom = nom;  
    }  
    public String getNom() {  
        return nom;  
    }  
}
```

- a) El constructor no pot rebre paràmetres
- b) L'atribut nom hauria de ser public
- c) El constructor assigna el paràmetre a si mateix i no modifica l'atribut nom
- d) getNom hauria de ser static

14. Quina opció respecta millor l'encapsulació?

- a) Atributs public i accés directe des de qualsevol classe
- b) Atributs private i mètodes public que controlen l'accés
- c) Atributs i mètodes static sense creació d'objectes
- d) Atributs sense modificador per simplificar l'ús



15. Què imprimeix aquest codi?

```
class Comptador {
    private static int total = 10;
    private int id;

    public Comptador() {
        total--;
        id = total;
    }
    public static int getTotal() {
        return total;
    }
    public int getId() {
        return id;
    }
}

class Main{
    public static void main(String args[]){
        Comptador a = new Comptador();
        Comptador b = new Comptador();
        System.out.println(a.getId() + "-" + Comptador.getTotal());
    }
}
```

- a) 1-2
- b) 9-8
- c) 8-8
- d) Error perquè total és private

16. Quin setter és més adequat per a un atribut private double preu que no pot ser negatiu?

- a) public void setPreu(double preu) { this.preu = preu; }
- b) public void setPreu(double preu) { if (preu >= 0) this.preu = preu; }
- c) public static void setPreu(double preu) { this.preu = preu; }
- d) private void setPreu(double preu) { preu = this.preu; }



RA5. Realitza operacions d'entrada i eixida d'informació

17		18		19		20	
----	--	----	--	----	--	----	--

17. Quina afirmació és correcta sobre una ruta relativa com "dades/entrada.txt"?

- a) Sempre apunta al mateix lloc en qualsevol ordinador
- b) Depén del directori de treball des d'on s'executa el programa
- c) Apunta a un lloc web d'internet precedit de https://
- d) Només es pot usar amb la classe File per a crear un objecte d'este tipus

18. Què retorna listFiles() quan s'aplica sobre un objecte File que representa un directori existent?

- a) Una cadena amb la ruta absoluta
- b) Un array amb fitxers i carpetes continguts en el directori
- c) Només els fitxers .txt de la carpeta
- d) Un ArrayList<String> amb els noms dels fitxers

19. Quina opció escriu text al final d'un fitxer sense esborrar el contingut anterior?

- a) `new FileReader(new FileWriter("log.txt"))`
- b) `new BufferedWriter(new FileWriter("log.txt", true))`
- c) `new BufferedWriter(new FileReader("log.txt", true))`
- d) `new FileWriter(new BufferedReader("log.txt"))`

20. Quin és el risc principal d'aquest codi?

```
Scanner sc = new Scanner(System.in);  
int edat = sc.nextInt();  
String nom = sc.nextLine();
```

- a) `nextInt()` tanca automàticament l'entrada
- b) `nextLine()` pot llegir el salt de línia pendent
- c) Scanner no pot llegir enters
- d) `nextLine()` converteix l'enter a text



RA6. Escriu programes que manipulen informació amb tipus avançats de dades

21		22		23		24	
----	--	----	--	----	--	----	--

21. Quina eixida produeix el codi?

```
int[][] v = {{2,4,6},{1,3,5},{7,7,7}};
int suma = 0;

for (int i = 0; i < v.length; i++) {
    for (int j = 0; j < v[i].length; j++)
        if (i == j)
            suma += v[i][j];
}

System.out.println(suma);
```

- a) 12
- b) 10
- c) 16
- d) 0

22. Quina afirmació és correcta sobre un array en Java?

- a) Pot canviar de mida automàticament
- b) Té una mida fixa després de crear-se
- c) Només pot guardar objectes, no tipus primitius
- d) Sempre comença en la posició 1

23. Què imprimeix aquest fragment?

```
ArrayList<String> noms = new ArrayList<>();
noms.add("Ada");
noms.add("Linus");
noms.add(1, "Grace");
System.out.println(noms.get(1));
```

- a) Ada
- b) Linus
- c) Grace
- d) Error perquè add no admet posició



24. Quina estructura seria més adequada per associar un DNI amb una persona concreta?

- a) Stack<Persona>
- b) Queue<Persona>
- c) HashMap<String, Persona>
- d) int[]

RA7. Desenvolupa programes aplicant característiques avançades de POO

25		26		27		28	
----	--	----	--	----	--	----	--

25. Que imprimirà el següent codi?

```
class Vehicle {
    public String arrancar() {
        return "Vehicle";
    }
}

class Cotxe extends Vehicle {
    @Override
    public String arrancar() {
        return "Cotxe";
    }
}

class App {
    public static void main(String args[]) {
        Cotxe c = new Vehicle();
        System.out.println(c.arrancar());
    }
}
```

- a) Vehicle
- b) Cotxe
- c) Error de compilació
- d) null

26. Quina afirmació és correcta sobre una interfície en Java?

- a) Permet definir un contracte que altres classes poden implementar
- b) Només pot usar-se si no existeix cap herència
- c) Tots els seus mètodes han de tindre obligatòriament codi complet
- d) Una classe que implementa una interfície deixa de poder tindre atributs propis



27. Quina afirmació sobre conceptes d'herència i polimorfisme és FALSA en Java?

- a) Un objecte serà de tots els tipus heretats i les interfícies que implementa que la seua classe
- b) Una classe pot heretar només d'una altra però implementar moltes interfícies
- c) Una classe abstracta ha de tindre com a mínim un mètode abstracte
- d) Una interfície només pot definir mètodes sense cos i constants

28. Quin fragment correspon a una sobreescriptura correcta?

- a)

```
class A { public void f(int x) {} }  
class B extends A { public void f(String x) {} }
```
- b)

```
class A { public void f() {} }  
class B extends A { @Override public void f() {} }
```
- c)

```
class A { private void f() {} }  
class B extends A { @Override private void f() {} }
```
- d)

```
class A { public static void f() {} }  
class B extends A { @Override public void f() {} }
```

RA9. Gestiona informació emmagatzemada en bases de dades

29		30		31		32	
----	--	----	--	----	--	----	--

29. Quin objecte s'utilitza habitualment per recórrer els resultats d'un SELECT en JDBC?

- a) Connection
- b) Statement
- c) ResultSet
- d) DriverManager

30. Quin mètode és adequat per executar una consulta SELECT?

- a) executeQuery()
- b) executeUpdate()
- c) insertRow()
- d) moveToInsertRow()

31. Quina forma és correcta per obtenir una connexió?

- a) DriverManager.getConnection(url, usuari, password)
- b) Connection.open(usuari, password, url)
- c) DriverManager.connect(usuari, url, password)
- d) new Connection(url, usuari, password)



32. Quina afirmació sobre Statement i createStatement() és correcta?

```
String sql = "SELECT * FROM alumnes WHERE nota > 8";  
Statement st = conn.createStatement();  
ResultSet rs = st.executeQuery(sql);
```

- a) createStatement() s'utilitza sobre una connexió oberta per obtenir un objecte capaç d'executar SQL
- b) createStatement() retorna directament un ResultSet amb totes les files d'una taula
- c) Statement només pot executar instruccions INSERT, però no consultes SELECT
- d) Statement s'utilitza per llegir fitxers de text línia a línia



PART PRÀCTICA:

- En la part pràctica es valorarà tant el funcionament com el disseny, la claredat, la utilització de les estructures de codi més adequades per a fer-lo reutilitzable i la documentació mínima del codi.
- No és necessari escriure els imports.
- Pots obviar els getters i setters que no s'hagen demanat explícitament.

Sistema de Bicicletes elèctriques "EcoBikePRG"

Una empresa tecnològica vol desenvolupar un sistema en Java per a gestionar una xarxa de punts intel·ligents de recàrrega de bicicletes elèctriques instal·lats en diferents municipis, detectant quins punts requereixen manteniment i deixant preparada la informació per als transportistes.

Exercici 1:

RA avaluats: RA1 - 2,67 punts · RA2 - 2,67 punts · RA4 - 8 punts · RA7 - 8 punts

L'empresa gestiona punts de recàrrega de bicicletes elèctriques i vol implementar una representació del sistema en Java. Cada punt ha de tindre un identificador, una ubicació i el nombre de recàrregues mensuals realitzades. L'identificador es una cadena de 5 posicions només serà vàlida si comença per les lletres PC; si no compleix aquestes condicions, al crear qualsevol punt de recàrrega nou s'haurà d'assignar automàticament l'identificador "PC000". Tots els punts han de poder calcular el seu cost de manteniment mensual, però el càlcul dependrà del tipus concret de punt. S'ha d'implementar el constructor.

Tenim dos tipus de punts de recàrrega:

1. Punts solars, que tenen un nombre de bicicletes en càrrega actual (0 com a mínim i 5 com a màxim). El cost de manteniment d'aquests punts es pot obtenir multiplicant en nombre de recàrregues mensuals per 0,35 €. S'ha d'implementar el constructor i el getter i el setter del nombre de bicicletes en càrrega respectant les restriccions del model.
2. Punts amb bateria externa, que han de guardar el percentatge de bateria restant. En aquest cas, a més del cost de les recàrregues, tenen un cost fix mensual afegit de 90 €. A més a més, hauran de poder-se marcar com a pendent de revisió quan la bateria restant siga inferior al 20%. Aquests punts son considerats revisables en el nostre sistema (el sistema ha de poder identificar si un punt es revisable o no) i tindran una funcionalitat de *ferRevisio* que implicarà recarregar la bateria al 100% i canviar la seua ubicació per una rebuda com a paràmetre, però només si estan marcats com a pendent de revisió. S'ha d'implementar el constructor, el getter de si està o no pendent de revisar i el setter de la bateria (que només podrà actualitzar-se si el nou valor de la bateria està entre 0 i 100).

Escriu el codi Java essencial per representar aquest model. Has de tindre en compte que **tots els getters i setters que necessiteu però que no es demanen explícitament es consideren ja implementats.**



Exercici 2:

RA avaluats: **RA1** - 2,67 punts · **RA2** - 2,67 punts · **RA3** - 4 punts · **RA5** - 8 punts · **RA6** - 8 punts

L'empresa guarda les dades dels punts de recàrrega en un fitxer de text situat dins d'una carpeta del projecte indicada mitjançant una ruta donada. El programa haurà de localitzar dins d'aquesta carpeta l'únic fitxer d'extensió ".txt" i llegir-ne el contingut línia a línia.

Cada línia representarà un punt de recàrrega i tindrà el format **id;ubicacio;recarreguesMensuals;bicicletesCarregant** per als punts solars, o bé **id;ubicacio;recarregues;bateria;perRevisar** per als punts amb bateria (teniu un exemple més endavant). Per tant, el tipus d'objecte que s'ha de crear s'haurà de deduir a partir del nombre de camps obtinguts.

A partir de cada línia vàlida, s'haurà de crear l'objecte corresponent del model de l'exercici anterior i guardar-lo en una cua per poder gestionar-lo posteriorment de manera ordenada. Els elements de la cua s'hauran de tractar de manera polimòrfica com a punts de recàrrega y el mètode haurà de controlar les excepcions que puguen aparèixer durant la localització del fitxer, i evitant que una excepció en les dades del fitxer (com la del **PC004**) interrompa tota la càrrega.

Exemple de fixer:

```
PC001;Plaça Major;120;4
PC002;Estació;80;15;true
PC003;Ajuntament;45;2
PC004;Avinguda;63;Església
PC005;Institut;60;75;false
```

Escriu el codi del mètode necessari per fer aquesta càrrega.

```
class GestorPunts {
    private Queue<Punt> cua = new LinkedList<>();
    public void carregarPunts(String ruta) {
        // codi de càrrega
    }
}
```



Exercici 3.

RA avaluats: **RA1** - 2,67 punts · **RA2** - 2,67 punts · **RA3** - 4 punts · **RA9** - 8 punts

A partir de la cua de punts de recàrrega creada en l'exercici anterior, l'empresa necessita generar la ruta de treball dels transportistes. El programa haurà de buidar la cua seleccionant únicament aquells que siguen punts revisables. D'aquests, només s'hauran de processar els que necessiten revisió.

El mètode preguntarà a l'usuari quina serà la nova ubicació després de fer la revisió; si aquesta ubicació passada a minúscules no és vàlida (no es troba a la llista *ubicacionsPermeses* rebuda com a paràmetre), s'haurà de llançar i capturar una excepció de tipus *UbicacioIncorrectaException* (que es considera ja creada) mostrar un missatge "Ubicació no permesa" i continuant amb el següent punt de la llista.

Cada punt seleccionat haurà de ser reubicat i guardat en una base de dades MySQL perquè els transportistes puguen consultar quins punts han de recollir i on han de portar-los. La taula ja existeix i s'anomena *ruta_transportistes*, amb les columnes *id_punt*, *ubicacio_actual* i *ubicacio_nova*. **Pots considerar implementats tots els getters necessaris.**

S'hauran de controlar també les excepcions derivades de l'accés a la base de dades, evitant que el programa finalitze bruscament.

Les credencials per a connectar-se a la base de dades son les següents:

- **URL:** jdbc:mysql://localhost:3306/recarrega_db
- **Usuari:** alumne
- **Contrasenya:** Recarrega26!

```
class GestorPunts {  
    private Queue<Punt> cua = new LinkedList<>();  
    private Scanner sc = new Scanner(System.in);  
    public void guardarRutaBateries(ArrayList<String> ubicacionsPermeses) {  
        // IMPLEMENTACIÓ  
    }  
}
```